

USB's CDC协议的OTA升级

USB's CDC协议的OTA升级

- 一 支持的软硬件平台
 - 1.1 支持的硬件平台
 - 1.2 支持的软件平台
- 二 OTA原理和介质介绍
 - 2.1 OTA简介和实现方法
 - 2.2 OTA升级支持的存储介质
 - 2.3 Freertos系统的ota功能使用流程
- 三 编译支持rtos_ota功能的SPL
 - 3.1 uboot添加支持rtos_ota功能
 - 3.1.1 MTD存储介质
 - 3.1.1.1 修改对应的板级配置文件
 - 3.1.1.2 编译uboot
 - 3.1.2 MMC存储介质
 - 3.1.2.1 修改对应的板级配置文件
 - 3.1.2.2 修改分区表
 - 3.1.2.3 编译uboot
- 四 编译带ota升级功能的zero.bin
 - 4.1 IConfigTool配置
 - 4.2 增加ota升级代码
 - 4.3 编译
- 五 烧录
 - 5.1 Freertos系统 作为主系统的烧录工具配置
 - 5.1.1 MTD存储介质
 - 5.1.2 MMC存储介质
 - 5.2 Liunx系统 作为主系统的烧录工具配置
 - 5.2.1 MTD存储介质
 - 5.2.2 MMC存储介质
- 六 设备端的OTA升级的流程
 - 6.1 ota的升级流程图
 - 6.2 ota升级过程中文件分包策略
 - 6.3 ota分区策略
 - 6.4 进入具有ota升级功能的Freertos系统
 - 6.4.1 主系统为Freertos系统
 - 6.4.2 主系统为Linux系统
 - 6.5 退出具有ota升级功能的Freertos系统
- 七 上位机的使用
- 八 ota的过程打印信息
 - 8.1 升级模式
 - 8.2 回读模式

一 支持的软硬件平台

1.1 支持的硬件平台

支持x1000、x1600、x2000、x2600、ad100系列CPU，已在ilock(x2000e & mmc0)、Halley6(x1600e & nand)、x2600_evb_v1.0(x2600e & nor)开发板上验证。

1.2 支持的软件平台

ota支持升级linux系统和Freertos系统。

二 OTA原理和介质介绍

2.1 OTA简介和实现方法

君正USB's CDC协议的OTA简单来说分为三个步骤。

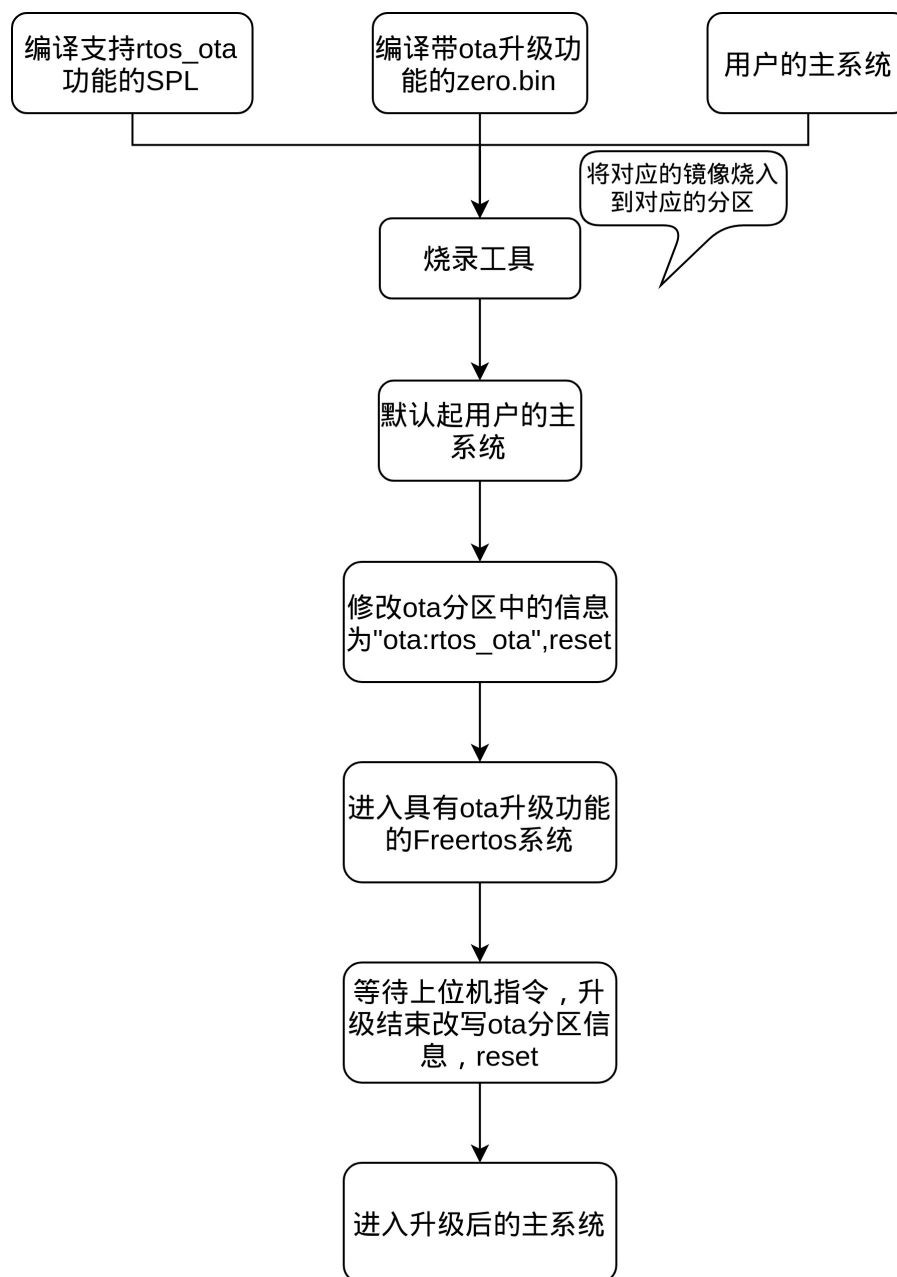
- 第一步：准备好需要更新的升级固件。
- 第二步：通过USB的CDC协议将升级固件传输到设备上。
- 第三步：设备将得到的OTA升级固件更新到自身的存储介质中，然后重启切换到最新升级的系统上，从而完成整个OTA流程。

君正基于Freertos系统，OTA升级的策略是采用创建一个rtos_ota分区，分区中放置具有ota升级功能的Freertos系统代码。SPL引导代码通过判断ota分区中的内容是否为"ota:kernel2"，来决定是否要进入具有ota升级功能的Freertos系统。也就是一台设备上具有ota升级功能的Freertos系统代码和用户的主系统，但是每次只运行其中的一套系统，两个系统共同使用一份SPL代码。

2.2 OTA升级支持的存储介质

君正USB's CDC协议的OTA目前支持的存储介质有mtd和mmc，mtd包含：nand和nor。

2.3 Freertos系统的ota功能使用流程



三 编译支持rtos_ota功能的SPL

3.1 uboot添加支持rtos_ota功能

3.1.1 MTD存储介质

以配备nor flash 的PD_X2600_EVB_V1.0开发板，主系统为Freertos的编译配置为例，编译需要支持rtos ota功能的uboot固件的步骤如下：

3.1.1.1 修改对应的板级配置文件

对应的SPL的编译配置为x2600e_base_rtos_sfc_nor，此配置对应的板级配置文件为x2600_base.h。

文件的路径如下：

[工程目录]/bootloader/uboot-x2000/include/configs/{board}.h

以x2600为例：[工程目录]/bootloader/uboot-x2000/include/configs/x2600_base.h

增加OTA的功能需要对x2600_base.h文件进行如下修改：

```
#ifndef __X2600_BASE_H__
#define __X2600_BASE_H__

#define CONFIG_ROOTFS_SQUASHFS
#define CONFIG_ROOTFS2_SQUASHFS
#define CONFIG_ARG_QUIET
#define CONFIG_SPL_SERIAL_SUPPORT

/* 添加以下宏定义 */
#define CONFIG_BOOT_RTOS_OTA

#include "x2600_base_common.h"

#endif /* __X2600_BASE_H__ */
```

uboot中与rtos的ota相关定义的文件路径如下：

[工程目录]/bootloader/uboot-x2000/include/configs/{soc}_base_common.h

以x2600为例：[工程目录]/bootloader/uboot-x2000/include/configs/x2600_base_common.h

对应的宏定义具体的解释如下：

```
#ifdef CONFIG_BOOT_RTOS_OTA
    /* 定义放置具有升级功能的rtos的分区名 */
    #define CONFIG_SPL_RTOS_OTA_NAME "rtos_ota"
#endif

#ifndef CONFIG_SPL_OTA_NAME
    /* 定义放置升级信息的分区名 */
    #define CONFIG_SPL_OTA_NAME "ota"
#endif

#ifndef CONFIG_SPL_RTOS_OTA_INFO
    /* 定义判断是否要进入rtos_ota分区的条件 */
    #define CONFIG_SPL_RTOS_OTA_INFO "ota:rtos_ota"
#endif

#ifdef CONFIG_SPL_RTOS_BOOT
#ifndef CONFIG_SPL_RTOS_NAME
    /* 若另一分区也为Freertos系统，且CONFIG_SPL_RTOS_NAME宏未定义，定义
CONFIG_SPL_RTOS_NAME为rtos */
    #define CONFIG_SPL_RTOS_NAME "rtos"
#endif
#endif /* CONFIG_SPL_RTOS_BOOT */

#endif /* CONFIG_BOOT_RTOS_OTA */
```

3.1.1.2 编译uboot

- 配置编译uboot需要的环境变量

通过以下命令，确定对应的芯片的uboot编译工具链：

```
#获取对应的芯片的uboot的编译工具链版本
fy@hlf:~/work/x2670/build$ grep -nr "uboot" configs/x2600e_nor_5.10_defconfig
3:APP_uboot_toolchain_dir=./tools/toolchains/mips-gcc720-glibc229
4:APP_uboot_dir=./bootloader/uboot-x2000
5:APP_uboot_config=x2600e_base_xImage_sfc_nor

#获取工具的绝对路径
fy@hlf:~/work/x2670/build$ cd ../tools/toolchains/mips-gcc720-glibc229
fy@hlf:~/work/x2670/tools/toolchains/mips-gcc720-glibc229$ pwd
/home/fy/work/x2670/tools/toolchains/mips-gcc720-glibc229

#回到对应的uboot代码的目录下
fy@hlf:~/work/x2670/tools/toolchains/mips-gcc720-glibc229$ cd
../../bootloader/uboot-x2000/

#配置编译uboot需要的工具链的路径，注意路径要链接到bin文件夹
fy@hlf:~/work/x2670/bootloader/uboot-x2000$ export
PATH=$PATH:/home/fy/work/x2670/tools/toolchains/mips-gcc720-glibc229/bin
```

- 编译对应的uboot配置

```
#清除之前make生成的所有文件
[工程目录]/bootloader/uboot-x2000$ make distclean
#编译对应的板级的uboot配置，此处以x2600e的nor flash的Freertos系统为例
[工程目录]/bootloader/uboot-x2000$ make x2600e_base_rtos_sfc_nor
```

编译成功后会在[工程目录]/bootloader/uboot-x2000/spl目录下生成目标文件：

```
fy@hlf:~/work/x2670/bootloader/uboot-x2000$ ls spl/ -l
total 344
drwxr-xr-x 3 fy fy 4096 6月 19 2023 arch
drwxr-xr-x 3 fy fy 4096 6月 19 2023 board
drwxr-xr-x 3 fy fy 4096 6月 19 2023 common
drwxr-xr-x 6 fy fy 4096 7月 11 2023 drivers
drwxr-xr-x 2 fy fy 4096 1月 31 13:54 lib
-rw-r--r-- 1 fy fy 8592 10月 24 16:35 Makefile
drwxr-xr-x 2 fy fy 4096 6月 19 2023 spl
-rwxr-xr-x 1 fy fy 220176 1月 31 13:54 u-boot-spl
-rwxr-xr-x 1 fy fy 13788 1月 31 13:54 u-boot-spl.bin
-rw-r--r-- 1 fy fy 617 1月 31 13:54 u-boot-spl.lds
-rw-r--r-- 1 fy fy 48473 1月 31 13:54 u-boot-spl.map
-rw-r--r-- 1 fy fy 24576 1月 31 13:54 u-boot-spl-pad.bin
```

3.1.2 MMC存储介质

以配备emmc的x2000_iLock_v1.0开发板，主系统为Freertos的编译配置为例，编译需要支持rtos ota功能的uboot固件的步骤如下：

3.1.2.1 修改对应的板级配置文件

对应的SPL的编译配置为x2000_base_rtos_mmc0，此配置对应的板级配置文件为x2000_base.h。

文件的路径如下：

```
[工程目录]/bootloader/uboot-x2000/include/configs/{board}.h
```

以x2000为例：[工程目录]/bootloader/uboot-x2000/include/configs/x2000_base.h

增加OTA的功能需要对x2000_base.h文件进行如下修改：

```
#ifndef __X2000_BASE_H__
#define __X2000_BASE_H__

#define CONFIG_ROOTFS_SQUASHFS
#define CONFIG_ROOTFS2_SQUASHFS
#define CONFIG_ARG_QUIET
#define CONFIG_SPL_SERIAL_SUPPORT

/* 添加以下宏定义 */
#define CONFIG_BOOT_RTOS_OTA

#include "x2000_base_common.h"

#endif /* __X2000_BASE_H__ */
```

uboot中与rtos的ota相关定义的文件路径如下：

```
[工程目录]/bootloader/uboot-x2000/include/configs/{soc}_base_common.h
```

以x2000为例：[工程目录]/bootloader/uboot-x2000/include/configs/x2000_base_common.h

对应的宏定义具体的解释如下：

```
#ifdef CONFIG_BOOT_RTOS_OTA
    /* 定义放置具有升级功能的rtos的分区名 */
    #define CONFIG_SPL_RTOS_OTA_NAME "rtos_ota"
#endif

#ifdef CONFIG_SPL_OTA_NAME
    /* 定义放置升级信息的分区名 */
    #define CONFIG_SPL_OTA_NAME "ota"
#endif

#ifdef CONFIG_SPL_RTOS_OTA_INFO
    /* 定义判断是否要进入rtos_ota分区的条件 */
    #define CONFIG_SPL_RTOS_OTA_INFO "ota:rtos_ota"
#endif

#ifdef CONFIG_SPL_RTOS_BOOT
#ifdef CONFIG_SPL_RTOS_NAME
    /* 若另一分区也为Freertos系统，且CONFIG_SPL_RTOS_NAME宏未定义，定义CONFIG_SPL_RTOS_NAME为rtos */
    #define CONFIG_SPL_RTOS_NAME "rtos"
#endif
```

```
#endif /* CONFIG_SPL_RTOS_BOOT */

#endif /* CONFIG_BOOT_RTOS_OTA */
```

3.1.2.2 修改分区表

分区表的路径如下：[工程目录]/bootloader/uboot-x2000/board/ingenic/芯片名_base/

eg: /home/fy/work/x2670/bootloader/uboot-x2000/board/ingenic/x2000_base

- 主系统为Freertos系统的分区表：

```
property:
    disk_size = 3728m
    gpt_header_lba = 512
    custom_signature = 0

partition:
    #name      = start,    size, fstype
    uboot      =      0m,      1m,
    rtos_ota   =      1m,      1m, EMPTY
    rtos       =      2m,      5m, EMPTY
    ota        =      7m,      1m, EMPTY

#fstype could be: LINUX_FS, FAT_FS, EMPTY
```

- 主系统为Linux系统的分区表：

```
property:
    disk_size = 3728m
    gpt_header_lba = 512
    custom_signature = 0

partition:
    #name      = start,    size, fstype
    uboot      =      0m,      1m,
    kernel     =      1m,      8m, EMPTY
    rootfs     =      9m,     50m, LINUX_FS
    rtos_ota   =     59m,      1m, EMPTY
    ota        =     60m,      1m, EMPTY

#fstype could be: LINUX_FS, FAT_FS, EMPTY
```

3.1.2.3 编译uboot

- 配置编译uboot需要的环境变量

通过以下命令，确定对应的芯片的uboot编译工具链：

```
#获取对应的芯片的uboot的编译工具链版本
fy@hlf:~/work/x2670/build$ grep -nr "uboot" configs/x2000_mmc0_defconfig
2:APP_uboot_toolchain_dir=../tools/toolchains/mips-gcc720-glibc229
3:APP_uboot_dir=../bootloader/uboot-x2000
4:APP_uboot_config=x2000_base_xImage_mmc0
```

```
#获取工具的绝对路径
fy@hlf:~/work/x2670/build$ cd ../tools/toolchains/mips-gcc720-glibc229
fy@hlf:~/work/x2670/tools/toolchains/mips-gcc720-glibc229$ pwd
/home/fy/work/x2670/tools/toolchains/mips-gcc720-glibc229

#回到对应的uboot代码的目录下
fy@hlf:~/work/x2670/tools/toolchains/mips-gcc720-glibc229$ cd
../../bootloader/uboot-x2000/

#配置编译uboot需要的工具链的路径，注意路径要到bin文件夹
fy@hlf:~/work/x2670/bootloader/uboot-x2000$ export
PATH=$PATH:/home/fy/work/x2670/tools/toolchains/mips-gcc720-glibc229/bin
```

- 编译对应的uboot配置

```
#清除之前make生成的所有文件
[工程目录]/bootloader/uboot-x2000$ make distclean

#编译对应的板级的uboot配置，此处以x2000e的mmc flash的Freertos系统为例
[工程目录]/bootloader/uboot-x2000$ make x2000_base_rtos_mmc0
```

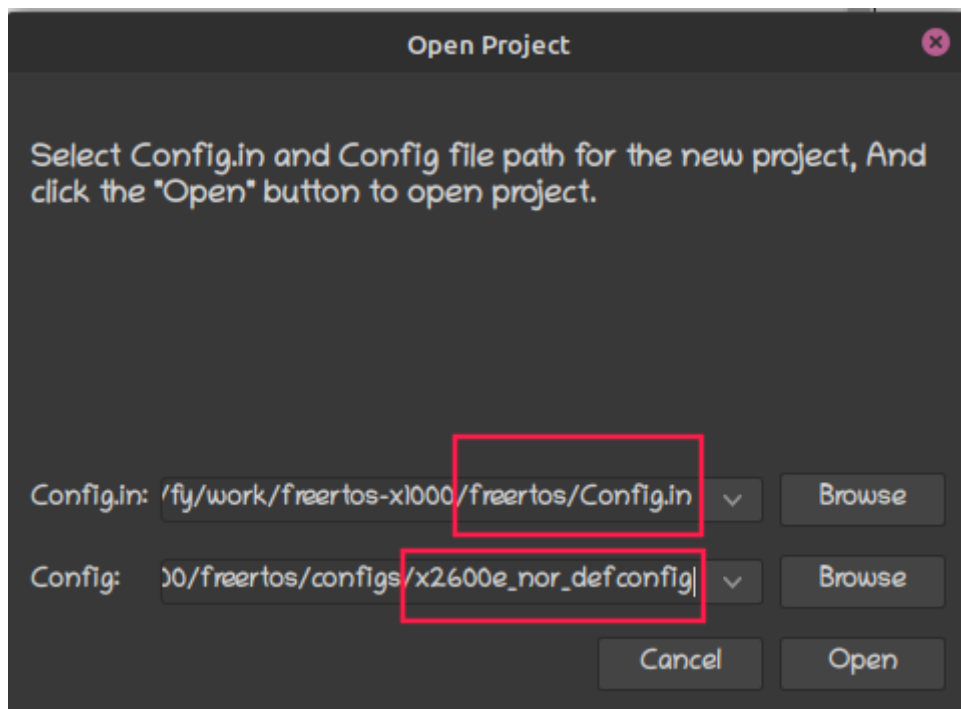
编译成功后会在[工程目录]/bootloader/uboot-x2000目录下生成目标文件：

```
fy@hlf:~/work/x2670/bootloader/uboot-x2000$ ls
api          disk         MAKEALL      rules.mk     u-boot.bin
arch         doc          Makefile     snapshot.commit u-boot.lds
board        drivers      make_spl     spl          u-boot.map
boards.cfg   dts          mkconfig     System.map   u-boot-spl-pad.bin_backup
check-rootfs_hash.patch examples      nand_spl     test         u-boot.srec
common       fs           net          test1        u-boot-with-spl.bin
config.mk    include      post         test.c       u-boot-with-spl-mbr-gpt.bin
COPYING      lib          README       tools        x2600_rtos_ota.patch
CREDITS      MAINTAINERS rtos_ota_update.patch u-boot
```

四 编译带ota升级功能的zero.bin

4.1 IConfigTool配置

以x2600e_nor_defconfig为例，打开IConfigTool工具，选择配置文件



勾选以下配置：

- 选中USB驱动





- 选中SFC驱动,若为MMC介质不需要选中



- 选中rtos的ota升级



- 选中watchdog (看门狗)



4.2 增加ota升级代码

修改vendor/vendor.c文件(以存储介质nor flash为例)：

```
#include <stdio.h>
#include "example/driver/ota_upgrade_nor_flash.c"//ota升级nor flash代码示例文件路径

void vendor_init(void *arg)
{
    ota_upgrade_nor_flash_test();
}
```

4.3 编译

- 首先先要配置环境变量，再使用对应板级和存储介质的配置文件，如x2600的nor flash：

```
fy@hlf:~/work/freertos-x1000/freertos$ source build/envsetup.sh
Set PATH success...
fy@hlf:~/work/freertos-x1000/freertos$ make x2600e_nor_defconfig
..writing .config.in
..writing include/config.h
```

- 配置之后就可以make：

```
fyehlf:~/work/freertos-x1000/freertos$ make
..include package/package.mk
..include package/build/build.mk
..include package/xburst2/xburst2.mk
..include package/xburst2/soc-x2600/soc-x2600.mk
..include package/lib/lib.mk
..include package/drivers/drivers.mk
..include package/drivers/usb/usb.mk
..include package/os/os.mk
..include package/os/freertos/freertos.mk
..include package/os/posix/posix.mk
..include package/filesystem/filesystem.mk
..include package/filesystem/flash/flash.mk
..include package/filesystem/elmfat/elmfat.mk
..include package/filesystem/debugfs/debugfs.mk
..include package/symbols/symbols.mk
..include package/shell/shell.mk
..include package/devices/devices.mk
..include package/devices/wireless/wireless.mk
..include package/vendor/vendor.mk
..include package/third_party/third_party.mk
..include package/newlib/newlib.mk
..include package/application/application.mk
lds.in build/build.lds.in
```

- 编译完成之后会在Freertos工程目录下生成zero.bin文件：

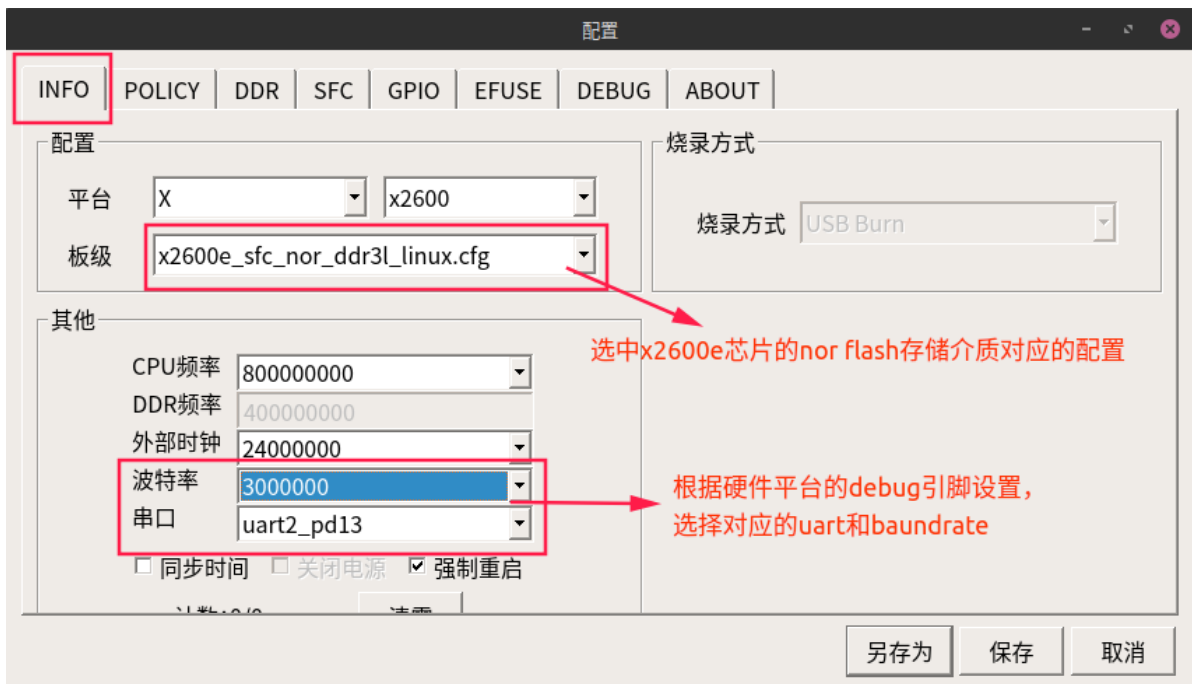
```
fyehlf:~/work/freertos-x1000/freertos$ ls
application  devices    include    newlib     pkt_send   pkt_send4   third_party  xburst2   zero.bin
build        drivers    lib         objs       pkt_send1  rtos-with-spl.bin  tools        zero0.bin  zero.dump.S
Config.in    example    Makefile    os         pkt_send2  shell        vendor        zero1.bin  zero.elf
configs      filesystem net         package    pkt_send3  symbols      xburst        zero2.bin
```

五 烧录

5.1 Freertos系统 作为主系统的烧录工具配置

5.1.1 MTD存储介质

此处以x2600e芯片，存储介质为nor flash为例：



配置

INFO | POLICY | DDR | **SFC** | GPIO | EFUSE | DEBUG | ABOUT

基本信息 | NOR信息 | 分区信息

基本信息

☒ 烧录参数到存储设备

SFC频率: 100000000

擦除

文件系统擦除块大小: 32768

擦除: 全部擦除

四线设置

☒ 烧录四线

☒ 启动四线

预留空间

☐ 激活SFC NAND预留空间

☐ 激活预留分区

另存为 | 保存 | 取消

配置

INFO | POLICY | DDR | **SFC** | GPIO | EFUSE | DEBUG | ABOUT

基本信息 | NOR信息 | **分区信息**

3 分区信息

分区大小可根据实际情况设置，注意分区不要重叠

	Partition name	offset	size	flags	Manage_mode
1	uboot	0x0	0x100000	0	MTD_MODE
2	rtos_ota	0x100000	0x100000	0	MTD_MODE
3	rtos	0x200000	0x500000	0	MTD_MODE
4	ota	0x700000	0x100000	0	MTD_MODE

分区名、flag、管理模式与此设置的一致

添加 | 修改 | 删除 | 上移 | 下移

另存为 | 保存 | 取消

配置

INFO | **POLICY** | DDR | SFC | GPIO | EFUSE | DEBUG | ABOUT

支持rtos_ota功能的SPL镜像

	label	type	ops	offset	attribute	setting
1	uboot	文件	SFC_NOR	0x0	boot-spl-pad.bin	...
2	rtos_ota	文件	SFC_NOR	0x100000	freertos/zero.bin	...
3	rtos	文件	SFC_NOR	0x200000	freertos/zero.bin	...
4	ota	文件	SFC_NOR	0x700000	000/freertos/null	...
5						
6						
7						
8						
9						
10						

type和ops与此一致

具有ota升级功能的Freertos系统

与SFC页的分区名和分区偏移一致

用户的主系统，此处以Freertos为例

存储ota升级相关的信息，初始烧录的文件可以为空

添加 | 删除 | 上移 | 下移

Nor ID:

参数偏移:

合并镜像 | 生成镜像

另存为 | 保存 | 取消

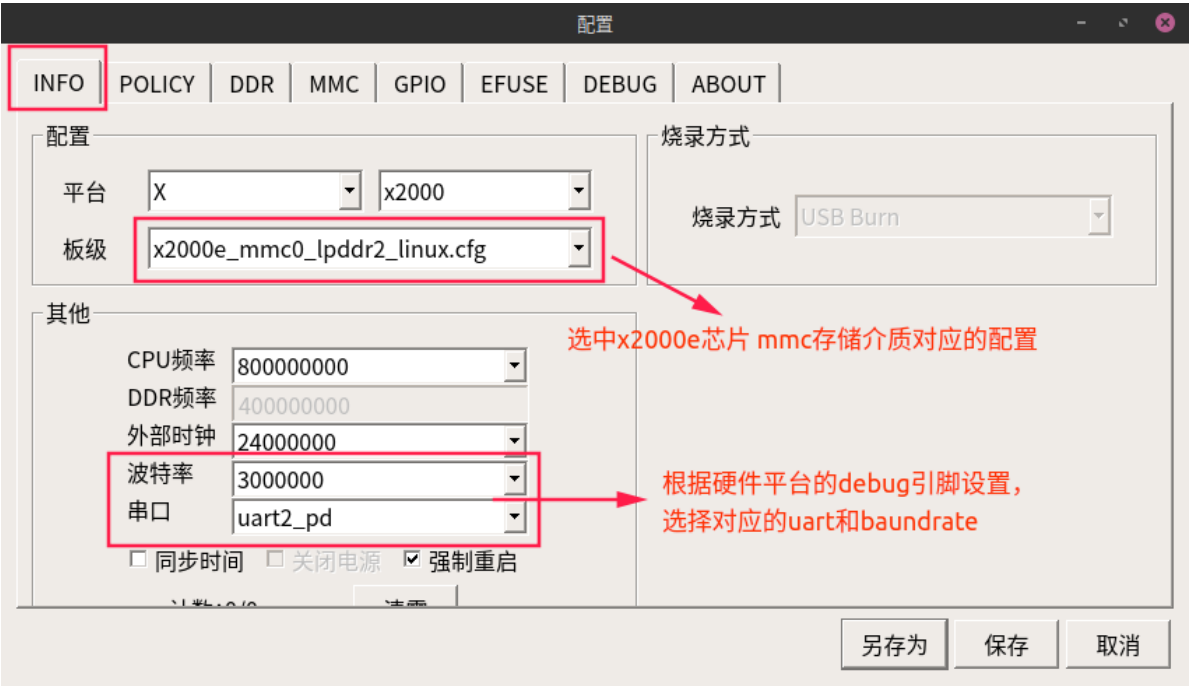
- uboot 分区：放置支持rtos_ota功能的SPL镜像
- rtos_ota 分区：放置具有ota升级功能的Freertos系统镜像
- rtos 分区：此处用Freertos系统作为示例，放置用户的主系统
- ota 分区：存储ota升级相关的信息，用于SPL做是否进入具有ota升级功能的Freertos系统的判断条件。ota分区烧录的文件可以为一个空文件。

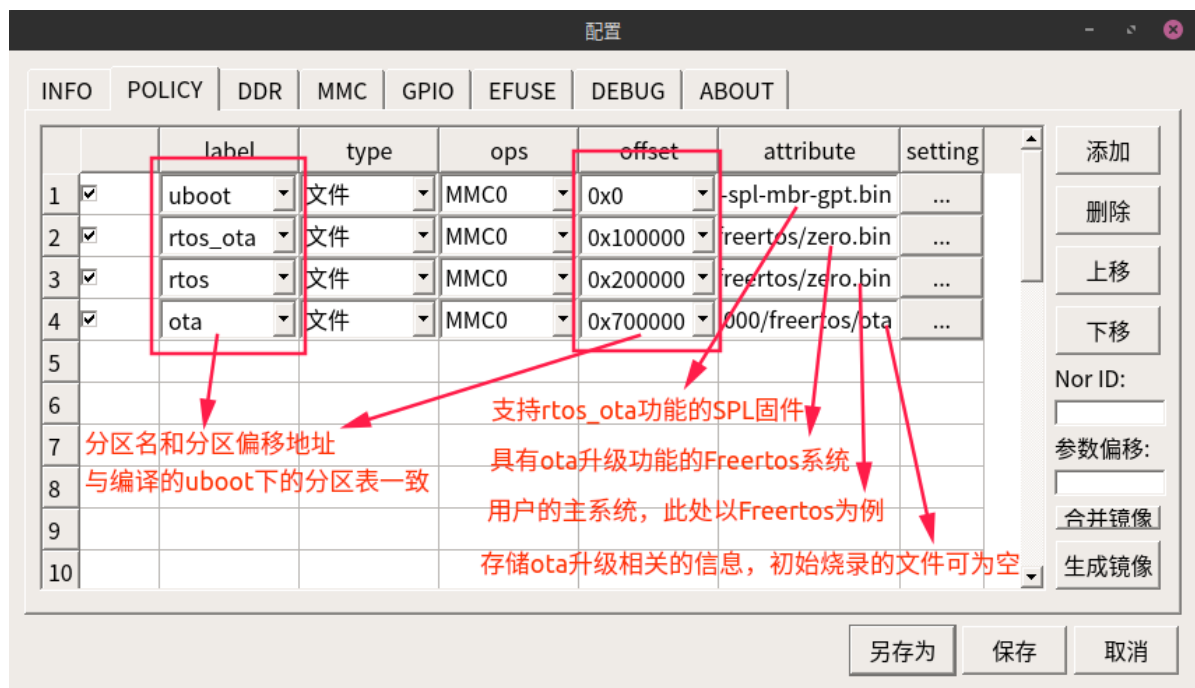
设置完成之后可以将此配置另存为配置文件。

注：分区的大小和分区的起始地址对齐flash的块大小。

5.1.2 MMC存储介质

此处以x2000e芯片，存储介质为mmc0为例：





- uboot 分区：放置支持rtos_ota功能的SPL镜像
- rtos_ota 分区：放置具有ota升级功能的Freertos系统镜像
- rtos 分区：此处用Freertos系统作为示例，放置用户的主系统
- ota 分区：存储ota升级相关的信息，用于SPL做是否进入具有ota升级功能的Freertos系统的判断条件。ota分区烧录的文件可以为一个空文件。

设置完成之后可以将此配置另存为配置文件。

注：分区的偏移量和分区名要与uboot中的分区表一致。

5.2 Liunx系统 作为主系统的烧录工具配置

5.2.1 MTD存储介质

此处以x2600e芯片，存储介质为nor flash为例：



配置

INFO | POLICY | DDR | **SFC** | GPIO | EFUSE | DEBUG | ABOUT

基本信息 | NOR信息 | 分区信息

基本信息

☒ 烧录参数到存储设备

SFC频率

擦除

文件系统擦除块大小

擦除

四线设置

☒ 烧录四线

☒ 启动四线

预留空间

☐ 激活SFC NAND预留空间

☐ 激活预留分区

另存为 保存 取消

配置

INFO | POLICY | DDR | **SFC** | GPIO | EFUSE | DEBUG | ABOUT

基本信息 | NOR信息 | **分区信息**

分区大小可根据实际需要设置，注意分区不要重叠

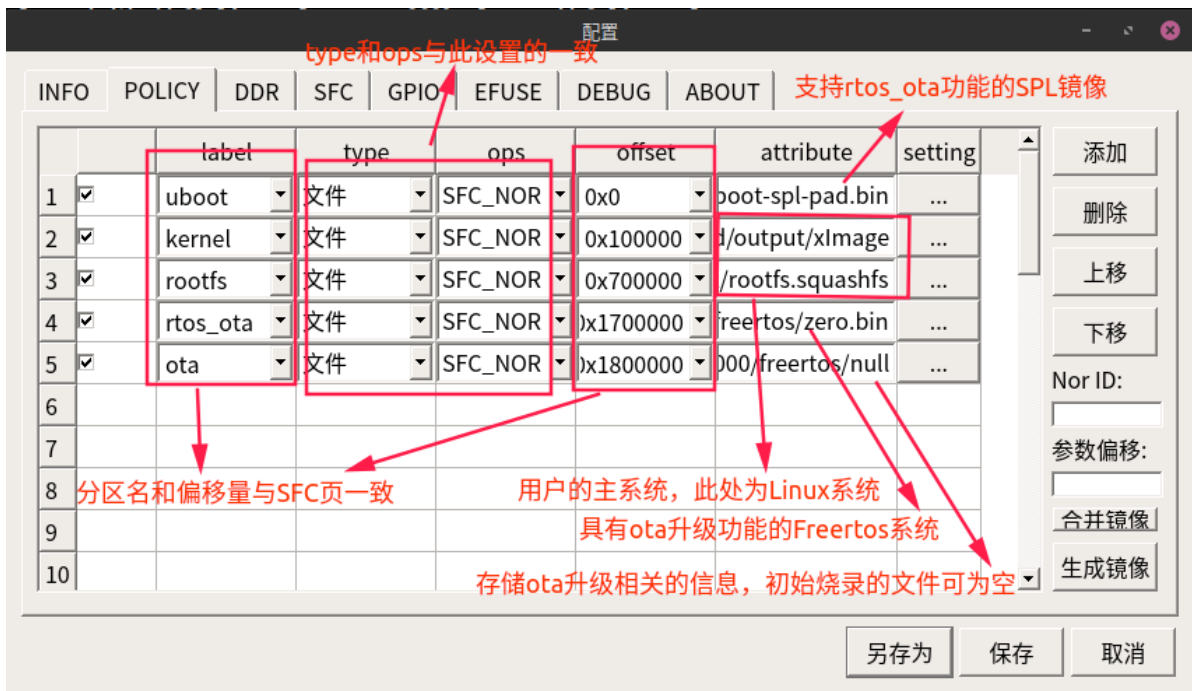
分区信息

	Partition name	offset	size	flags	Manage_mode
1	uboot	0x0	0x100000	0	MTD_MODE
2	kernel	0x100000	0x600000	0	MTD_MODE
3	rootfs	0x700000	0x1000000	0	MTD_MODE
4	rtos_ota	0x1700000	0x100000	0	MTD_MODE
5	ota	0x1800000	0x100000	0	MTD_MODE

添加 修改 删除 上移 下移

分区名、flags、管理模式与此设置的一致

另存为 保存 取消



- uboot 分区：放置支持rtos_ota功能的SPL镜像
- rtos_ota 分区：放置具有ota升级功能的Freertos系统镜像
- kernel 分区：用户的主系统，此处为Linux系统，放置kernel镜像
- rootfs分区：用户的主系统，此处为Linux系统，放置rootfs镜像
- ota 分区：存储ota升级相关的信息，用于SPL做是否进入具有ota升级功能的Freertos系统的判断条件。ota分区烧录的文件可以为一个空文件。

设置完成之后可以将此配置另存为配置文件。

注：建议将Linux系统的分区放置在rtos_ota之前，分区的大小和分区的起始地址对齐flash的块大小。

5.2.2 MMC存储介质

此处以x2000e芯片，存储介质为mmc0为例：



配置

INFO | POLICY | DDR | **MMC** | GPIO | EFUSE | DEBUG | ABOUT

强制擦除

☒ 烧录前擦除

☒ 全擦

☐ 部分擦除

	从	到
1		
2		
3		
4		
5		

增强分区设置

☐ 用户增强分区

☐ 最大增强分区大小

☐ 添加用户增强分区

	起始 KB	长度 KB
1		

☐ 添加通用增强分区

	长度 KB	属性类型	属性
1		扩展	Default

另存为 保存 取消

配置

INFO | POLICY | DDR | **MMC** | GPIO | EFUSE | DEBUG | ABOUT

		label	type	ops	offset	attribute	setting
1	☒	uboot	文件	MMC0	0x0	-spl/mbr-gpt.bin	...
2	☒	kernel	文件	MMC0	0x100000	l/output/xImage	...
3	☒	rootfs	文件	MMC0	0x900000	/rootfs.squashfs	...
4	☒	rtos_ota	文件	MMC0	0x3B00000	reertos/zero.bin	...
5	☒	ota	文件	MMC0	0x3C00000	000/reertos/null	...
6							
7							
8							
9							
10							

支持rtos功能的SPL固件

具有ota升级功能的Freertos系统

用户的主系统，此处以Linux为例

存储ota升级相关的信息，初始烧录的文件可为空

分区名和分区偏移地址与编译的uboot下的分区表一致

添加 删除 上移 下移

Nor ID:

参数偏移:

合并镜像 生成镜像

另存为 保存 取消

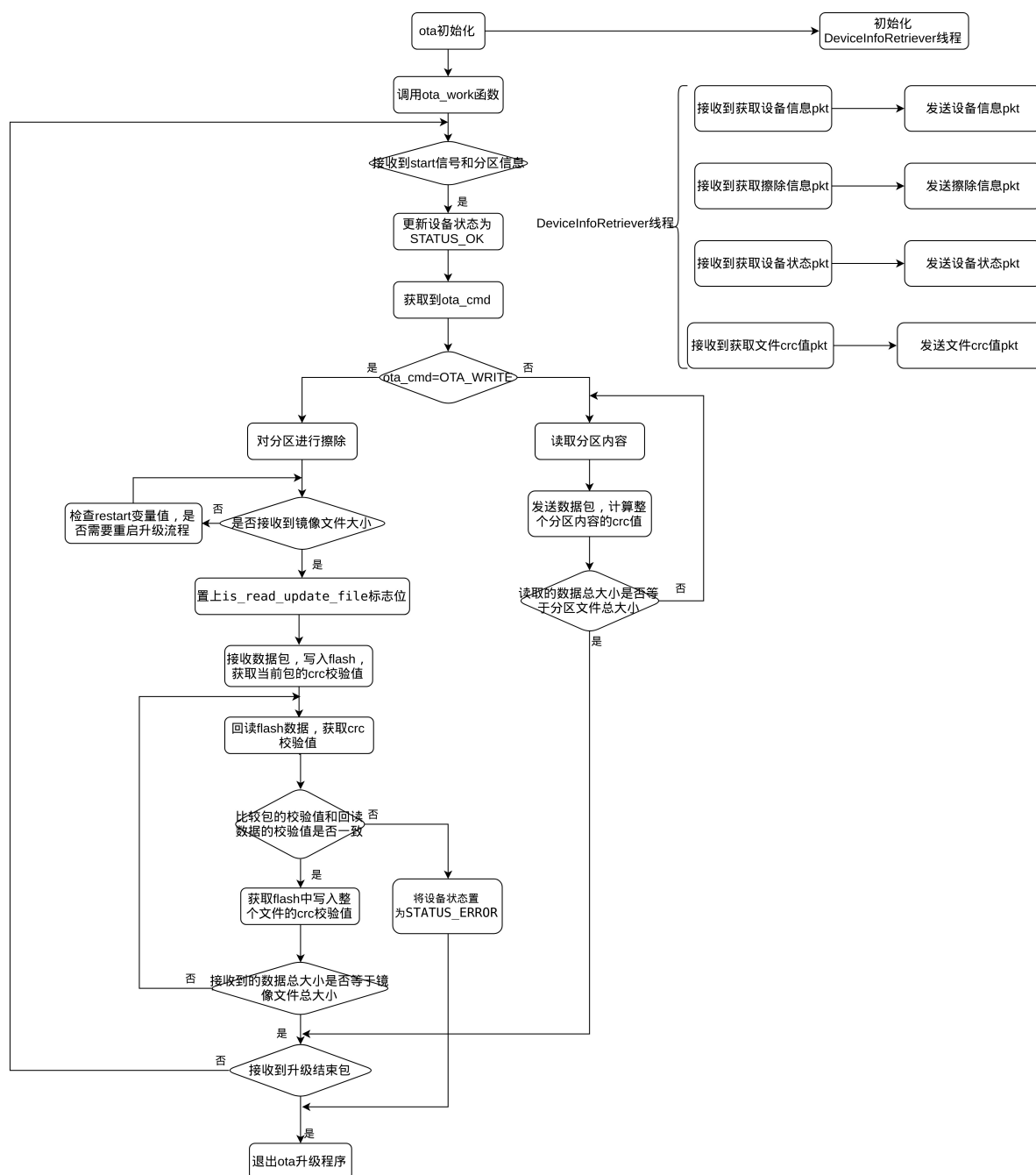
- uboot 分区：放置支持rtos_ota功能的SPL镜像
- rtos_ota 分区：放置具有ota升级功能的Freertos系统镜像
- kernel 分区：用户的主系统，此处为Linux系统，放置kernel镜像
- rootfs分区：用户的主系统，此处为Linux系统，放置rootfs镜像
- ota 分区：存储ota升级相关的信息，用于SPL做是否进入具有ota升级功能的Freertos系统的判断条件。ota分区烧录的文件可以为一个空文件。

设置完成之后可以将此配置另存为配置文件。

注：建议将Linux系统的分区放置在rtos_ota之前，分区的偏移量和分区名要与uboot中的分区表一致。

六 设备端的OTA升级的流程

6.1 ota的升级流程图



6.2 ota升级过程中文件分包策略

在通过usb的cdc协议传输镜像的过程中，数据包大小是以存储介质的页大小为单位进行分包传输。传输过程中的每个包和包头都进行crc32校验，并将结果存储在包头中，设备端和上位机对包进行接收时，会对包内容和包头进行校验。

每次写入到存储介质中的数据都会进行回读，并计算其crc32校验值，与数据包中包含的crc32校验值进行比较。

6.3 ota分区策略

更新分区中的镜像时，由上位机告知设备端对哪个分区进行升级，升级结束也由上位机告知设备端将ota分区中的信息改写。

不建议对uboot分区进行升级，一旦升级失败机器将无法启动。

目前采取的更新策略是由rtos_ota分区中的镜像对各个分区进行升级，因此若对rtos_ota分区进行升级，升级失败可能会导致rtos_ota分区中的镜像无法启动，无法使用ota升级功能。

6.4 进入具有ota升级功能的Freertos系统

修改ota分区中的内容为"ota:rtos_ota"

6.4.1 主系统为Freertos系统

Freertos系统作为主系统可使用以下命令修改ota分区中的信息：

```
# nor flash存储介质可用以下命令
$ nor_write 0x700000 0x6F 0x74 0x61 0x3A 0x72 0x74 0x6F 0x73 0x5F 0x6F 0x74 0x61
$ reset
$ nor_read 0x700000 0x20
read_buf:
0x6f 0x74 0x61 0x3a 0x72 0x74 0x6f 0x73 0x5f 0x6f
0x74 0x61 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff
0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff
0xff 0xff

# nand flash存储介质可用以下命令
$ nand_write 0x700000 0x6F 0x74 0x61 0x3A 0x72 0x74 0x6F 0x73 0x5F 0x6F 0x74 0x61
$ reset
$ nand_read 0x700000 0x20
read_buf:
0x6f 0x74 0x61 0x3a 0x72 0x74 0x6f 0x73 0x5f 0x6f
0x74 0x61 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff
0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff
0xff 0xff

# mmc flash存储介质可用以下命令
$ mmc_write 0x700000 0x6F 0x74 0x61 0x3A 0x72 0x74 0x6F 0x73 0x5F 0x6F 0x74 0x61
$ reset
$ mmc_read 0x700000 0x20
read_buf:
0x6f 0x74 0x61 0x3a 0x72 0x74 0x6f 0x73 0x5f 0x6f
0x74 0x61 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00
```

6.4.2 主系统为Linux系统

```
$ cd /tmp
$ echo "ota:rtos_ota" > ota
#of对应的路径为ota分区所对应的设备名，此处为mmc设备
$ dd if=./ota of=/dev/mmcblk0p4 count=512 bs=1
#of对应的路径为ota分区所对应的设备名，此处为mtd设备
$ dd if=./ota of=/dev/mtdblk0p4 count=512 bs=1
```

做完以上操作，重启即可进入具有ota升级功能的Freertos系统。

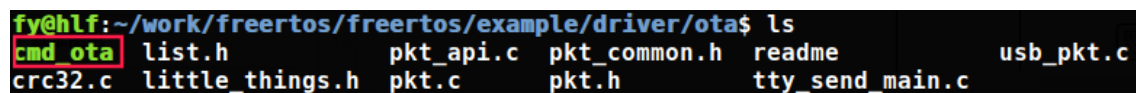
6.5 退出具有ota升级功能的Freertos系统

修改ota分区中的内容不为"ota:rtos_ota"之后重启即可。

七 上位机的使用

上位机的代码位于Freertos工程下，目录为：[Freertos工程目录]/freertos/example/driver/ota

具体文件如下图：



```
fy@hl1:~/work/freertos/freertos/example/driver/ota$ ls
cmd_ota  list.h      pkt_api.c  pkt_common.h  readme      usb_pkt.c
crc32.c  little_things.h  pkt.c      pkt.h          tty_send_main.c
```

cmd_ota为编译生成的可执行文件。

在上述目录下，编译和使用上位机的代码：

```
#用gcc编译链编译脚本
$ gcc crc32.c pkt_api.c pkt.c tty_send_main.c usb_pkt.c -lpthread -o cmd_ota
#*****具体使用*****
#对指定分区进行ota升级
#./cmd_ota dev=[ota通讯的串口] update=[升级/回读(1/0)] file=[升级的文件名] partition=
[升级的分区名]
$ ./cmd_ota dev=/dev/ttyACM0 update=1 file=zero.bin partition=rtos
#回读分区
#./cmd_ota dev=[ota通讯的串口] update=[升级/回读(1/0)] file=[回读的内容存放的文件路径]
partition=[回读的分区名]
$ ./cmd_ota dev=/dev/ttyACM0 update=0 file=test.bin partition=rtos
#修改ota分区的信息，不为ota:rtos_ota，这里以ota:rtos为例。
#进行此操作之后，复位即可退出具有ota升级功能的Freertos系统。
$ echo "ota:rtos" > ota
$ ./cmd_ota dev=/dev/ttyACM0 file=ota partition=ota
# 复位芯片
$ ./cmd_ota dev=/dev/ttyACM0 reset

# 若ota后续还有其他需执行的代码，则可使用以下命令退出ota流程（执行后面的代码）
# 使用此命令之后，会退出ota流程，注销掉cdc的设备节点，上位机ota命令不可用
$ ./cmd_ota dev=/dev/ttyACM0 exit
```

八 ota的过程打印信息

8.1 升级模式

上位机：

```
fy@hlf:~/work/freertos/freertos/example/driver/ota$ ./cmd_ota dev=/dev/ttyACM0 update=1 file=zero.bin partition=rtos
file_len : 536692 升级文件的总长度
dev status: 0 设备状态，为0标识STATUS_OK
pagesize: 2048 设备的页大小，上位机的数据包按照页大小为单位分包
partition_size : 5242880 升级分区的总大小
partition_blocks: 40 升级分区的总块数
waiting flash erase partition. : 40 / 40 擦除进度
sending: 536692 FileVerify: a53a7f67 发送文件的总长度 和文件的校验值
pri_data.FileVerify: a53a7f67 接收到设备发送过来的文件的校验值
```

设备端：

```
[0.635032] usb gadget serial connect!!!
[0.635195] serial_connect_callback 1
[0.663867] usb serial: dwDTERate 9600, bCharFormat 0, bParityType 0, bDataBits 8
[2.189035] usb serial: dwDTERate 57600, bCharFormat 0, bParityType 0, bDataBits 8
[12.042038] usb serial: dwDTERate 115200, bCharFormat 0, bParityType 0, bDataBits 8
[74.114998] ota:file len : 536692 需要接收的文件长度
ota: receiving : 536692 接收到的文件长度
[74.290030] file verify : a53a7f67 接收到的文件的校验值
[74.290151] ota: update or read partition success!
[74.290323]
```

8.2 回读模式

上位机：

```
fy@hlf:~/work/freertos/freertos/example/driver/ota$ ./cmd_ota dev=/dev/ttyACM0 update=0 file=test.bin partition=rtos
dev status: 0 设备状态，为0表示设备状态为STATUS_OK
pagesize: 2048 设备的页大小，上位机的数据包按照页大小为单位分包
partition_size : 5242880 回读分区的总大小
partition_blocks: 40 回读分区的总块数
reading: 5242880 FileVerify: fd fb7445 回读的数据长度和整个数据长度的校验值
pri_data.FileVerify: fd fb7445 从设备端获取的校验值
```

设备端：

```
$
$
ota: sending 5242880 发送数据的总长度
[803.727484] data->file_verify : fd fb7445 数据的校验值
[803.727627] ota: update or read partition success!
[803.727804]
```